ProcessLight: Process Supervision for Large Language Model Based Traffic Signal Control

Shuaitao Zhao, Tianlong Zhou, Weijie Wang, Jiasheng Shi, Weixiong Rao*
School of Computer Science and Technology, Tongji University, Shanghai, China
{shuaitaozhao,wxrao}@tongji.edu.cn

Abstract

Large Language Models (LLMs) have recently been introduced into traffic signal control (TSC) as decision agents due to their strengths in human-readable reasoning generation. Yet, existing LLM TSC methods optimize only from final outcomes and fail to distinguish valid from flawed reasoning steps, causing useful or misleading steps to be jointly updated and thus impairing the model's learning of effective reasoning. To bridge this gap, we propose an LLM-based framework ProcessLight to decompose signal decisions into verifiable semantic steps. Building on ProcessLight, we further develop Step-wise Traffic Process Policy Optimization (STeP-PO), a novel reinforcement learning framework that optimizes structured reasoning processes through step-level credit assignment. Specifically, STeP-PO uses step quality scores to evaluate local reasoning quality and step importance to measure each step's influence on the final action, and then assigns step-level advantages over a semantic step tree structure. The resulting step-level advantages are propagated to reasoning tokens, enabling fine-grained policy optimization beyond outcome-only rewards. Extensive experiments over multiple real-world datasets demonstrate the superiority of our methods. Our code is available at https://github.com/wenzhaoabc/processlight.

1 Introduction

Rapid urbanization has led to a substantial increase in urban population, making traffic congestion an increasingly severe challenge for modern cities (Wei et al., 2019c; Wu et al., 2023; Koonce et al., 2008; Di et al., 2019). Traffic signal control, as a core mechanism for regulating vehicle flows at intersections, has long been a central research problem in intelligent transportation systems.

Existing TSC methods have evolved from traditional controllers to deep reinforcement learning approaches. Rule-based controllers, such as fixed-time control and pressure-based control, are computationally efficient and highly interpretable. Yet, they heavily depend upon manually designed priors, and struggle to adapt to complex and dynamic traffic conditions (Oroojlooy et al., 2020; Wei et al., 2019a; Liang et al., 2022; Wu et al., 2023). Reinforcement learning(RL) based methods learn adaptive signal control policies from feedback provided by traffic simulators (Wei et al., 2018, 2019a,b; Oroojlooy et al., 2020; Zhang et al., 2022; Wu et al., 2023; Yu et al., 2020). Despite promising performance, these methods are trained on compact numerical states and scalar rewards, which limits their interpretability and generalization ability in diverse traffic scenarios.

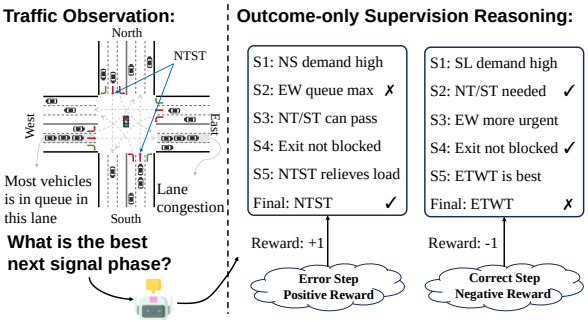


Figure 1: Outcome-level supervision cannot effectively distinguish correct and incorrect reasoning steps in the traffic signal reasoning process, weakening the model's ability to learn correct reasoning.

Recently, large language models (LLMs) have been introduced into traffic signal control to exploit their logical reasoning ability and natural-language generalization capability. Existing studies show that LLMs can transform traffic observations into interpretable control decisions (Lai et al., 2025; Yuan et al., 2026; Zou et al., 2026). However, current LLM-based TSC methods mainly optimize models according to final actions or overall trajectory outcomes, while largely overlooking the intermediate reasoning steps. As a result, they cannot

*Corresponding author.

effectively distinguish correct reasoning steps from erroneous ones. Erroneous intermediate reasoning may still be reinforced by a positive trajectory-level reward when the final action happens to be correct, thereby misleading the model to learn unreliable reasoning patterns.

As illustrated in Figure 1, even when a reasoning trajectory contains *incorrect* intermediate steps, the entire trajectory may still be reinforced when the final control decision is correct. In contrast, when the final decision is incorrect, the whole reasoning process is penalized, even though some intermediate traffic judgments may be valid and useful. This outcome-only optimization paradigm provides coarse and potentially biased supervision, making it difficult to determine which reasoning steps should be reinforced, corrected, or suppressed.

To bridge this gap, we propose **ProcessLight**, an LLM-based traffic signal control framework with explicit step-wise reasoning. By decomposing each traffic signal decision into a sequence of reasoning steps, ProcessLight can provide verifiable traffic state evidence for each step, and thus improves the interpretability of LLM-based signal control. More importantly, it provides fine-grained process-level information for reasoning supervision.

We further propose **STeP-PO**, a Step-wise Traffic Process Policy Optimization algorithm for optimizing ProcessLight. STeP-PO consists of three key components. First, it evaluates the quality of each reasoning step based on traffic-grounded claims by combining semantic soft matching with observation-based validity verification. Second, it estimates step importance through counterfactual sensitivity, where each reasoning step is removed and the resulting change in the final action distribution is measured. Third, it constructs a semantic step tree that merges semantically equivalent reasoning steps under the same preceding logic and assigns comparable hierarchical advantages to different steps. These advantages are further propagated to the token level within the reasoning text, enabling fine-grained policy optimization beyond reliance on outcome-only rewards. In summary, we make the following contributions.

- We propose **ProcessLight**, an LLM-based traffic signal control framework that formulates TSC as a process-supervised reasoning problem, enabling fine-grained optimization of intermediate traffic judgments.
- We introduce **STeP-PO**, a step-wise policy optimization algorithm that integrates step-quality evaluation and step-importance estimation to provide step-level supervision for LLM-based traffic signal control.
- We conduct extensive experiments in representative traffic simulation scenarios, demonstrating the effectiveness of the proposed method.

2 Related Work

2.1 Traffic Signal Control

Early methods of traffic signal control, e.g., fixed-time control and MaxPressure, are simple and stable, but their hand-crafted rules make them less adaptive to time-varying traffic demand and network-level coordination (Koonce et al., 2008; Varaiya, 2013; Qadri et al., 2020; Wei et al., 2019c). Deep reinforcement learning (DRL) methods improve adaptivity by learning signal policies from simulation. Representative studies include PressLight, which integrates pressure-based control into RL (Wei et al., 2019a), CoLight, which models network cooperation with graph attention (Wei et al., 2019b), and MPLight, which scales decentralized DRL to large road networks (Chen et al., 2020). However, these neural controllers remain limited by black-box decision processes, weak interpretability, and unstable generalization under distribution shifts. Recently, LLM-based controllers have introduced language reasoning into TSC. LLMLight treats LLMs as traffic signal agents (Lai et al., 2025), CoLLMLight studies cooperative LLM agents for network-wide control (Yuan et al., 2026), and Traffic-R1 applies two-stage RL to reduce the gap between reasoning and deployment (Zou et al., 2026). These methods mainly optimize policies using final control outcomes or action-level feedback, leaving intermediate reasoning steps weakly supervised.

2.2 RL Fine-tuning for LLMs

RL fine-tuning is widely used to improve instruction following and reasoning. RLHF typically optimizes LLMs with PPO (Christiano et al., 2017; Ouyang et al., 2022; Schulman et al., 2017), but the used critic for advantage estimation suffers from high memory cost and training instability. DPO bypasses explicit reward modeling by directly optimizing preference pairs (Rafailov et al., 2023), while GRPO removes the critic through group-relative baselines is effective for reasoning-oriented training (Shao et al., 2024; Guo et al., 2025a).

Nevertheless, trajectory-level rewards cannot distinguish the quality or contribution of individual reasoning steps within the same response. Recent credit-assignment methods, e.g., VinePPO and SPO, address this limitation with finer-grained advantage estimation (Kazemnejad et al., 2025; Guo et al., 2025b). In contrast, ProcessLight introduces step-level reward design and credit assignment for TSC reasoning, enabling direct supervision of how an LLM derives traffic-signal actions rather than only whether the final action succeeds.

3 Problem Definition

In this section, we introduce the key concepts of traffic signal control and formulate the LLM-based traffic signal control problem studied in this paper. More details on intersections, movements and signal phases are provided in the Appendix A.1.

Definition 1 ***Road Network.*** *The road network is a directed graph composed of intersections I and roads E. Each road consists of multiple lanes L, and each lane corresponds to a movement at the intersection.*

Definition 2 ***Traffic Signal Control.*** *At each signal-switching time step, the agent selects one phase a from a predefined phase set $\mathcal{A} = \{a_1, \ldots, a_m\}$. Each phase corresponds to a group of non-conflicting movements.*

We formulate traffic signal control as a partially observable Markov decision process. Each intersection is controlled by an LLM agent with policy π_θ. At each signal-switching time step, the agent observes traffic conditions $\mathcal{O}$. Given the task description $\mathcal{D}_{\text{task}}$ and reasoning step guide $\mathcal{G}$, the LLM agent generates a step-by-step reasoning trajectory Y and an executable phase action $a \in \mathcal{A}$ from the predefined phase set $\mathcal{A}$ by the policy π_θ:

$$(Y, a) = \pi_\theta\left(\text{Prompt}(\mathcal{O}, \mathcal{D}_{\text{task}}, \mathcal{G}, \mathcal{A})\right). \quad (1)$$

Here, the reasoning trajectory Y contains K steps $(s^1, \ldots, s^K)$. The objective of our task is to optimize the policy π_θ to improve transportation efficiency through high-quality reasoning.

4 Methodology

In this section, we first present an overview of ProcessLight, and then introduce the proposed Step-wise Traffic Process Policy Optimization (STeP-PO) that is used to optimize ProcessLight.

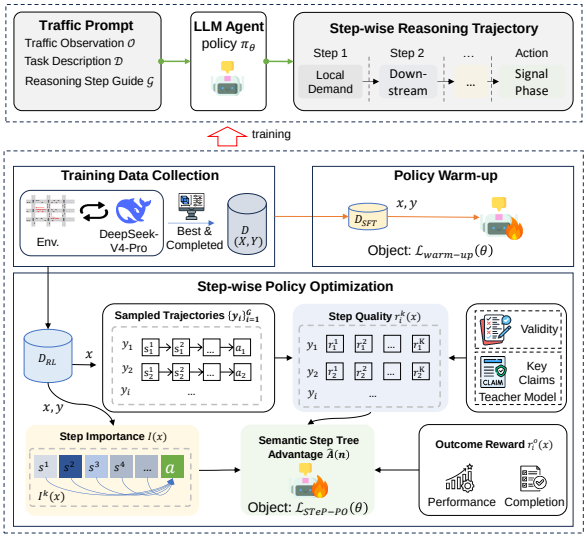


Figure 2: Overview of ProcessLight with two stages, (1) task prompt construction and (2) step-wise reasoning. The proposed STeP-PO algorithm is used for policy optimization.

4.1 ProcessLight Framework

In Figure 2, the workflow of ProcessLight consists of two stages: (1) prompt construction and (2) step-wise reasoning and action execution.

Prompt Construction. The prompt contains three main parts: current traffic observation $\mathcal{O}$, task description $\mathcal{D}_{\text{task}}$ and reasoning step guide $\mathcal{G}$. The observation $\mathcal{O}$ contains queue lengths and vehicle counts of the signalized and neighboring intersections, together with short-term historical trends for spatiotemporal reasoning. The guide $\mathcal{G}$ defines the reasoning direction and step decomposition, requiring the LLM to examine the current phases, local demand, downstream congestion, neighboring coordination, traffic trends, and phase fairness before selecting the final signal control action. We provide prompt examples in AppendixA.13.

Step-wise Reasoning and Action Execution. Given the prompt X, ProcessLight performs step-wise reasoning by following the number K of predefined decision steps. At each step s^k with $k \in [1, K]$, the LLM agent generates both the reasoning evidence and the corresponding step-level conclusion, and then gives the final signal control action a. Hence, ProcessLight not only executes effective traffic control decisions a, but also provides fine-grained reasoning evidence for each decision step s^k. This step-wise reasoning process improves interpretability and, more importantly, provides the necessary step-level information for subsequent reasoning process supervision.

4.2 STeP-PO

4.2.1 Overview

This section introduces the proposed Step-wise Traffic Process Policy Optimization (STeP-PO). STeP-PO contains three stages: (1)*Training Data Collection*, (2)*Policy Warm-up* and (3)*Step-wise Policy Optimization*. We provide the overall algorithm in Appendix A.4.

4.2.2 Training Data Collection

We begin by collecting diverse traffic scenarios from the traffic simulator. For each scenario, we evaluate all candidate phase actions for the target intersection with a fixed-horizon network-wide simulation, while other intersections follow the same baseline control policy, and select the action that yields the best traffic performance, e.g., lowest average queue length. Then, we prompt a large language model (e.g. DeepSeek-V4-Pro in Figure 2) using the prompt construction process described in Section 4.1 to generate step-wise reasoning trajectories aligned with the best action and the required output format. The resulting dataset $\mathcal{D}$ consists of input prompts X paired with step-wise reasoning trajectories Y. A small subset $\mathcal{D}_{\mathrm{SFT}} \subseteq \mathcal{D}$ is used for supervised warm-up, while the remaining data $\mathcal{D}_{\mathrm{RL}}$ for step-wise policy optimization.

4.2.3 Policy Warm-up

To equip the policy model with traffic-aware reasoning and format-following capabilities, we introduce a supervised warm-up stage before reinforcement learning (RL). The warm-up fine-tuning is performed on the above subset $\mathcal{D}_{\mathrm{SFT}}$ with the following objective:

$$\mathcal{L}_{\mathrm{warm-up}}(\theta) = -\mathbb{E}_{(X,Y)\sim\mathcal{D}_{\mathrm{SFT}}}\left[\sum_{t=1}^{|Y|}\log \pi_\theta\left(Y_t \mid X, Y_{<t}\right)\right] \tag{2}$$

4.2.4 Step-wise policy optimization

Outcome-only optimization is performed on entire reasoning trajectories with trivial guidance on intermediate steps. STeP-PO addresses this issue by optimizing the policy π_θ on $\mathcal{D}_{\mathrm{RL}}$ with a step-wise objective $\mathcal{L}_{\mathrm{STeP\text{-}PO}}(\theta)$. As shown in Fig. 2, STeP-PO involves three components: *step quality*, which measures whether an intermediate process is locally beneficial, *step importance*, estimating the step contribution to the final action, and a *semantic step tree*, where shared prefix steps are merged across trajectories into a single tree node. Hence, each node is assigned with a step-level advantage with a low variance. Step-level advantage is then assigned to the tokens within each step to derive token-level advantages for policy optimization.

Step Quality. Step quality helps to provide fine-grained supervision for intermediate reasoning steps, enabling STeP-PO to distinguish reliable local decisions from noisy or hallucinated ones. Specifically, for each prompt $X \in \mathcal{D}_{\mathrm{RL}}$, the policy model π_θ samples G reasoning trajectories $Y = \{y_i\}_{i=1}^{G}$, where each trajectory y_i is decomposed into K steps, i.e., $y_i = (s_i^1, \ldots, s_i^K)$. To evaluate the decision quality of each step, we require the LLM to output step-level decision claims. Each claim is a fine-grained and verifiable traffic judgment grounded in the current observation, such as "[ETWT][Queue Length] > [NTST][Queue Length]" or "MAX[AWT][PHASE]=NLSL". For the k-th step each training sample y_i, a teacher model pre-extracts the reference key-claim set, denoted by $\mathcal{C}_i^k$, to serve as the criterion for step-quality evaluation.

To estimate the step quality score, we use a soft key-claim matching method inspired by work (Zhang et al., 2025) and a traffic observation-based verification. Specifically, for the k-th step of trajectory y_i and the associated claim set C_i^k, we first use a validity gate $\mathrm{Valid}(\cdot)$ to check whether all generated claims are supported by the current traffic observation. If any claim is contradicted by the observation, the step quality score is assigned -1; otherwise, its reward is given by the matching score against the reference key claims:

$$r_i^k = \begin{cases} -1, & \text{if } \mathrm{Valid}(C_i^k) = \text{False}, \\ m_i^k, & \text{otherwise}, \end{cases} \tag{3}$$

where $m_i^k = |C_{i,\mathrm{match}}^k|/|\mathcal{C}_i^k|$ and $C_{i,\mathrm{match}}^k$ denotes the subset of key-claims matched by the generated claims. This design penalizes hallucinated traffic judgments, supports semantically equivalent claim forms, and prevents reward inflation from irrelevant claims. It thereby provides a reliable step quality signal for step-wise policy optimization.

Step Importance. To assess the marginal effect of each reasoning step on the final decision, we incorporate counterfactual sensitivity into STeP-PO for step importance estimation. For each training pair $(x, y) \in \mathcal{D}_{\mathrm{RL}}$ with $y = (s^1, \ldots, s^K)$, we remove the k-th step and construct the counterfactual trajectory $y^{-k} = (s^1, \ldots, s^{k-1}, s^{k+1}, \ldots, s^K)$. We further map each phase action to a dedicated

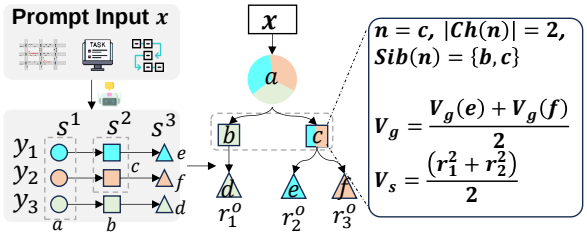


Figure 3: Semantic step tree construction. Reasoning steps with soft-matched claims are merged into the same node. The global state value is recursively propagated from child nodes, while the local step-level state value is computed from the step quality rewards.

action token (e.g., ETWT is presented as a signal token), such that phase probability can be directly obtained from next-token prediction probabilities.

We denote the phase distribution conditioned on the complete trajectory y as $p^0(\cdot)$ and the counterfactual distribution conditioned on y^{-k} as $p^{-k}(\cdot)$. The distributional discrepancy between $p^0(\cdot)$ and $p^{-k}(\cdot)$ measures the sensitivity of the final phase decision to step s^k. We employ D_{JSD} (Jensen–Shannon divergence) to quantify the divergence between distributions:

$$b^k(x) = D_{\mathrm{JSD}}\left(p^0(\cdot), p^{-k}(\cdot)\right). \tag{4}$$

Then we perform normalization over trajectory steps to derive the relative importance:

$$I^k(x) = \frac{b^k(x)}{\sum_{u=1}^{K} b^u(x) + \epsilon}, \tag{5}$$

where the small constant ϵ prevents numerical instability. This normalization captures the relative effect of each reasoning step within the trajectory y. Thus, it allows outcome feedback to be concentrated on those steps that are more decisive for the final phase action. Practically, these importance scores are pre-computed per prompt before policy optimization, and then reused across all associated rollouts. Thus, no additional forward or backward passes are required during RL training.

Reward Design. We define the outcome and process rewards. The outcome reward r_i^o consists of accuracy reward r_i^{acc} and reasoning completion reward r_i^{com}. The former reward r_i^{acc} evaluates whether the model selects the best phase action, while the latter one r_i^{com} checks whether the model produces all required reasoning steps in the prescribed format. We denote $r_i^o = r_i^{\mathrm{acc}} + r_i^{\mathrm{com}}$. For process-level supervision, we directly use the step quality score of each step as the step reward.

Semantic Step Tree Advantage. After evaluating step quality and deriving step importance with STeP-PO, we assign step-level advantages to strengthen supervision for intermediate reasoning. To this end, we adopt a tree-based structure to preserve prefix-conditioned dependencies during LLM reasoning and then to establish a fair local baseline for cross-step comparison. In Figure 3, we take the input x as the root node, and then merge those reasoning steps that are located at the same reasoning position across different trajectories and have semantically-matched claim sets. Next, we attach each trajectory's final action as a leaf node. Detailed algorithm for how to construct the semantic step tree is provided in Appendix A.5.

For a certain node n within this tree structure, we denote its sibling nodes and child nodes by $\mathrm{Sib}(n)$ and $\mathrm{Chd}(n)$, respectively, and the set of reasoning steps contained in node n by $\mathcal{S}(n)$. Next, we define the state value of node n as $V(n)$, which consists of two parts: the global state value $V_{\mathrm{g}}(n)$, which indicates its contribution to the final outcome, and the step-level state value $V_{\mathrm{s}}(n)$, which measures the intrinsic quality of the corresponding reasoning step. Specifically, we recursively compute the global state value by bottom-up aggregation:

$$\hat{V}_g(n) = \begin{cases} r_i^{\mathrm{o}}, & \text{leaf node,} \\ \frac{1}{|\mathrm{Chd}(n)|} \sum_{n' \in \mathrm{Chd}(n)} \hat{V}_g(n'), & \text{otherwise.} \end{cases} \tag{6}$$

For the step-level state value, we aggregate the step quality rewards of all reasoning steps merged into the same node:

$$\hat{V}_{\mathrm{s}}(n) = \frac{1}{|\mathcal{S}(n)|} \sum_{s_i^k \in \mathcal{S}(n)} r_i^k. \tag{7}$$

We then estimate the node-level advantage by jointly considering the global advantage and the step-level advantage. The global and step advantage are both computed with a normalized local baseline over sibling nodes, for $\ell \in \{\mathrm{g}, \mathrm{s}\}$:

$$\hat{A}_\ell(n) = \frac{\hat{V}_\ell(n) - \mathrm{mean}_{n' \in \mathrm{Sib}(n)} \hat{V}_\ell(n')}{\mathrm{std}_{n' \in \mathrm{Sib}(n)} \hat{V}_\ell(n') + \epsilon}, \tag{8}$$

where ϵ is a small constant for numerical stability. Then we get global advantage $\hat{A}_g(n)$ and local advantage $\hat{A}_s(n)$. The final node-level advantage is then defined as

$$\hat{A}(n) = w^k(x) \hat{A}_{\mathrm{g}}(n) + \lambda \hat{A}_{\mathrm{s}}(n), \tag{9}$$

where λ is a hyperparameter controlling the contribution of the step-level advantage. The weight

$w^k(x)$ is derived from the normalized step importance $I^k(x)$ to preserve the average scale of the global credit, where η controls the strength of importance weighting.

$$w^k(x) = (1-\eta) + \eta K I^k(x). \tag{10}$$

Unlike the conventional outcome-level advantage, the proposed tree-structured advantage above constructs comparable sibling node groups under shared reasoning prefixes and jointly considers local relative advantages according to step-level decision quality. In this way, STeP-PO alleviates the credit assignment bias caused by relying solely on overall reasoning outcomes, while providing fine-grained step-level optimization signals that explicitly preserve the contextual dependencies of preceding reasoning steps.

4.2.5 Policy Optimization Objective

We assign $\hat{A}(n)$ to all tokens within the corresponding reasoning-step segment and denote $\hat{A}_{i,t}$ as the advantage assigned to the t-th token in the i-th sampled trajectory y_i. Given a group of G rollouts sampled from the old policy $\pi_{\theta_{\text{old}}}$, we define the token-level importance sampling ratio and its clipped form as

$$\begin{aligned} \rho_{i,t}(\theta) &= \frac{\pi_\theta(y_{i,t} \mid x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x, y_{i,<t})}, \\ \bar{\rho}_{i,t}(\theta) &= \text{clip}(\rho_{i,t}(\theta), 1-\epsilon, 1+\epsilon). \end{aligned} \tag{11}$$

The final STeP-PO objective is defined as

$$\begin{aligned} \mathcal{L}_{\text{STeP-PO}}(\theta) = -\mathbb{E}_{x,\{y_i\}_{i=1}^{G}} \Bigg[\frac{1}{G} \sum_{i=1}^{G} \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \\ \Big(\min\left(\rho_{i,t}(\theta)\hat{A}_{i,t}, \bar{\rho}_{i,t}(\theta)\hat{A}_{i,t}\right) - \beta D_{\text{KL}}^{i,t} \Big) \Bigg], \end{aligned} \tag{12}$$

where β controls the KL regularization strength. Following GRPO-style optimization, the token-level KL penalty is estimated by

$$D_{\text{KL}}^{i,t} = \frac{\pi_{\text{ref}}(y_{i,t} \mid x, y_{i,<t})}{\pi_\theta(y_{i,t} \mid x, y_{i,<t})} - \log \frac{\pi_{\text{ref}}(y_{i,t} \mid x, y_{i,<t})}{\pi_\theta(y_{i,t} \mid x, y_{i,<t})} - 1. \tag{13}$$

5 Experiments

5.1 Experimental Setup

5.1.1 Datasets

Datasets. We train our model on one synthetic dataset (Synthetic-1) and evaluate its zero-shot generalization ability on five real datasets(Jinan1, Jinan2, Jinan3, Hangzhou1, Hangzhou2) (Wei et al., 2019c). Statistics for these datasets are detailed in Table 1

Table 1: Statistics of datasets.

Flow dataset	Structure	Vehicles	Arrival rate (vehicles/5min)			
			Mean	Std	Max	Min
Jinan 1	3×4	6295	523.67	98.52	671	255
Jinan 2		4365	362.83	74.81	493	236
Jinan 3		5494	456.92	46.38	544	362
Hangzhou 1	4×4	2983	247.67	40.44	332	211
Hangzhou 2		6984	581.08	318.43	1145	202
Synthetic 1	4×4	8000	666.6	32.10	735	612

- **Jinan**: A dataset from the Dongfeng subdistrict in Jinan, China, consisting of 12 intersections with three traffic flow datasets. Every intersection is composed of two 400-meter east-west roads and two 800-meter north-south roads.
- **Hangzhou**: A dataset from the Gudang subdistrict in Hangzhou, China, consisting of 16 intersections with two traffic flow datasets. Every intersection is composed of two 800-meter east-west roads and two 600-meter north-south roads.
- **Synthetic-1**: A synthetic dataset consisting of 16 intersections and each road between two intersections is 300 meters long.

5.1.2 Environment Settings

We conduct experiments on the open-source microscopic traffic simulator CityFlow (Zhang et al., 2019). Each intersection adopts four signal phases, and right-turn traffic is always allowed. The green phase lasts 15 seconds, followed by a 3-second yellow light and a 2-second all-red interval. All traffic flow scenarios are simulated for one hour.

5.1.3 Compared Methods

We compare our method with three types of methods, traditional transportation, RL-based and LLM-based works. Traditional transportation methods include FixedTime (Koonce et al., 2008) and MaxPressure (Varaiya, 2013) as baselines. For RL-based methods, we compare with eight methods, including MPLight (Chen et al., 2020), CoLight (Wei et al., 2019b), AttendLight (Oroojlooy et al., 2020), EfficientPressLight (Wu et al., 2021), EfficientMPLight (Wu et al., 2021), EfficientCoLight (Wu et al., 2021), AdvancedMPLight (Zhang et al., 2022) and AdvancedCoLight (Zhang et al., 2022). For LLM-based methods, we compare with LLMLight (Lai et al., 2025), CoLLMLight (Yuan et al., 2026) and Traffic-R1 (Zou et al., 2026). Additionally,

Table 2: Zero-shot traffic control performance on real-world traffic flows. Lower values indicate better performance. The best results are in bold and the second-best results are underlined.

Method	Jinan 1			Jinan 2			Jinan 3			Hangzhou 1			Hangzhou 2		
	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓
Traditional Methods															
FixedTime	410.07	157.00	190.36	521.91	232.12	405.89	463.07	184.99	282.31	564.91	204.65	169.57	553.78	137.69	267.12
MaxPressure	305.15	59.93	72.67	318.12	77.98	136.36	298.02	62.23	94.96	343.97	47.08	39.01	438.54	102.06	197.99
RL Methods															
EfficientPressLight	316.85	77.93	136.27	310.08	66.08	80.12	302.16	67.20	102.56	472.23	178.99	148.31	502.59	172.35	334.37
EfficientCoLight	310.23	70.10	122.57	298.99	54.05	65.54	293.58	57.86	88.30	340.52	43.85	36.34	419.34	82.13	159.34
AttendLight	557.35	292.16	510.88	443.58	193.23	234.29	531.52	269.35	411.06	486.69	182.19	150.96	560.49	161.61	313.53
EfficientMPLight	311.41	71.43	124.91	301.97	57.28	69.45	293.78	58.04	88.58	384.45	93.46	77.44	433.5	92.14	178.76
AdvancedCoLight	298.81	60.40	105.61	**292.99**	**49.01**	**59.41**	286.18	51.26	78.23	328.58	32.95	**27.30**	407.93	76.50	148.42
MPLight	466.00	214.83	375.65	384.79	138.23	167.61	421.48	185.16	282.57	558.10	234.91	194.65	550.89	151.67	294.24
AdvancedMPLight	313.15	75.18	131.47	298.2	54.13	65.64	290.16	55.36	84.49	372.22	80.23	66.48	437.29	102.24	198.34
CoLight	478.90	224.42	392.42	407.75	158.29	191.93	429.4	188.05	286.98	522.52	209.19	173.34	535.23	145.58	282.42
LLM-based Methods															
LightGPT-3B	406.13	160.26	194.32	411.45	173.48	303.34	402.70	167.71	255.95	412.47	117.12	97.05	503.97	137.13	266.03
LightGPT-8B	343.65	100.06	121.32	358.39	120.48	210.67	340.19	105.97	161.72	376.63	85.02	70.45	440.57	111.78	216.86
LLMLight	297.61	52.53	63.69	310.50	70.21	122.77	292.00	55.88	85.28	336.63	40.71	33.73	421.34	86.08	166.99
CoLLMLight	298.99	54.35	65.90	311.23	71.06	124.25	292.25	56.58	86.35	336.25	41.30	34.22	419.96	74.02	**143.59**
Traffic-R1	299.75	54.44	66.01	313.04	72.73	127.19	293.71	57.65	87.98	334.89	38.44	31.85	421.31	80.66	156.49
ProcessLight (ours)															
Qwen2.5-3B	327.90	82.95	100.58	356.37	117.97	206.28	327.34	92.23	140.76	353.87	59.15	49.01	438.37	109.85	213.11
Qwen3-8B	306.93	71.67	84.08	313.57	72.76	127.23	292.57	56.62	86.40	338.84	41.51	34.40	426.83	86.76	168.32
Llama3.1-8B	317.47	80.05	94.84	339.83	115.69	189.50	327.03	89.59	128.86	324.80	56.14	42.87	413.31	97.75	202.24
Qwen3-32B	299.67	54.37	65.92	312.16	71.67	125.33	293.96	57.85	88.28	338.05	40.87	33.86	426.62	85.51	168.88
DeepSeek-V4-Flash	296.83	50.22	61.96	302.12	68.51	116.28	286.17	55.31	83.09	321.98	41.64	34.51	416.45	82.60	164.13
DeepSeek-V4-Pro	290.93	51.33	60.88	301.53	70.53	114.83	285.12	56.60	82.43	322.37	41.01	33.98	412.11	81.91	165.58
ProcessLight	**289.34**	**45.03**	**55.94**	293.18	54.16	102.67	**284.33**	**50.97**	**78.20**	**314.99**	**32.82**	30.48	**407.24**	**73.13**	154.21

we assess the performance of general LLMs integrated within our ProcessLight framework, including Llama, Qwen, and DeepSeek series. More details refer to Appendix A.2.

5.1.4 Evaluation Metrics

We adopt three widely used traffic signal control metrics including *Average Travel Time (ATT)*, *Average Waiting Time (AWT)*, and *Average Queue Length (AQL)*. Their lower values indicate better traffic efficiency and control performance. More details about these metrics refer to Appendix A.3.

5.1.5 Implementation Details

We implement ProcessLight with Qwen2.5-3B (Yang et al., 2025) as the LLM-agent and DeepSeek-V4-Pro (DeepSeek-AI, 2026) as the teacher model, and split the collected dataset into two subsets: $\mathcal{D}_{\mathrm{SFT}}$ with 500 samples and $\mathcal{D}_{\mathrm{RL}}$ with 3000 samples. For fairness, all LLM baselines are trained and evaluated with the same experimental environment and configurations. Since Traffic-R1 (Zou et al., 2026) has not released its official source code, we directly adopt its publicly available model weights from HuggingFace for evaluation[1].

[1] https://huggingface.co/Season998/Traffic-R1

5.2 Performance Comparison

We evaluate ProcessLight on conventional traffic signal control tasks using five real-world traffic flows from two China cities (Jinan and Hangzhou). Following the cross-domain evaluation protocol, all learning-based methods are trained on Synthetic-1 and then directly tested on unseen real-world traffic flows without further fine-tuning. As shown in Table 2, ProcessLight achieves the best overall performance among all compared methods, obtaining the best results on most metrics. It also consistently outperforms existing LLM-based traffic control methods on almost all metrics, demonstrating that optimizing the reasoning process provides a more effective control policy than relying only on prompted reasoning or outcome-level optimization.

Without any fine-tuning, the smaller Qwen2.5-3B surpasses LightGPT-8B across five datasets. This demonstrates that model performance is not exclusively determined by parameter scale and fine-tuning; the stepwise reasoning mechanism of ProcessLight also contributes. By compelling LLMs to generate explicit reasoning steps, ProcessLight enables LLM agents to comprehensively analyze core traffic control factors and make rational intermediate decisions, effectively boosting traffic control performance prior to policy optimization.

Table 3: Ablation study on five real-world datasets.

Variant	Average Travel Time (s) ↓				
	Jinan 1	Jinan 2	Jinan 3	Hangzhou 1	Hangzhou 2
Qwen2.5-3B	327.90	356.37	327.34	353.87	438.37
+ Warm-up	320.91	344.55	319.17	345.61	432.92
+ SQ	300.64	315.18	298.33	330.99	418.24
+ SI	293.23	312.87	292.18	318.72	409.82
+ SSTA (ProcessLight)	**289.34**	**293.18**	**284.33**	**314.99**	**407.24**

Table 4: Effectiveness of process-level supervision.

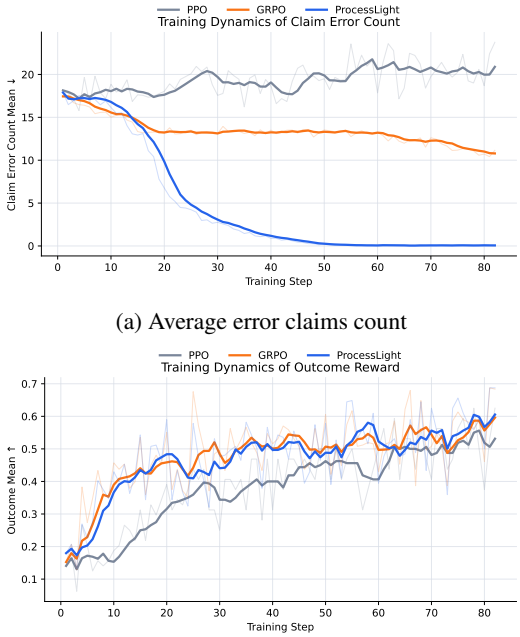
Training Method	Average Travel Time (s) ↓	
	Jinan 1	Hangzhou 1
Warm-up only	321.23	352.12
Warm-up + Outcome-level Reward	305.74	334.25
Warm-up + Step-wise Reward	293.64	321.47
Warm-up + STeP-PO	**289.34**	**314.99**

5.3 Ablation Study

We perform progressive ablation studies on five real-world traffic datasets with Qwen2.5-3B as the backbone to investigate the effect of four modules: the policy warm-up stage and three core designs: step quality reward (SQ), step importance (SI), and semantic step tree advantage (SSTA). Specifically, SQ introduces averaged step-level quality rewards into final outcome rewards, SI reallocates global advantages across reasoning steps via normalized importance weights, and SSTA builds tree-based semantic advantages to instantiate the full STeP-PO paradigm. Table 3 demonstrate consistent performance improvements via progressive module integration. The warm-up stage efficiently helps the LLM grasp basic traffic reasoning capabilities. SQ incentivizes valid reasoning steps, SI prioritizes critical reasoning steps, and SSTA enables fair advantage assignment via a tree-based structure. The full STeP-PO obtains the best performance, validating the synergistic effect of all three modules.

5.4 Effectiveness of Process Supervision

To verify how process-level supervision provides effective optimization signal for LLM-based traffic signal control, we compare outcome-only RL with our step-wise process optimization. Specifically, we evaluate four settings on Jinan1 and Hangzhou1 using ATT as the metric: (1) *Warm-up only*, where the model is trained only by supervised fine-tuning; (2) *Warm-up + Outcome-level Reward*, where the model is optimized by the final outcome reward only; (3) *Warm-up + Step-wise Reward*, where the model is further guided by the proposed step quality reward; and (4) our *STeP-PO*. In Table 4, both outcome-level reward and process-level supervision improve over the warm-up model, while our



(a) Average error claims count

(b) Average outcome reward

Figure 4: Training dynamics of reward-side indicators, (a) average error claims count and (b) average outcome reward.

STeP-PO performs best. This further demonstrates that step-wise policy optimization is more effective in enhancing LLM reasoning capabilities, as it provides more fine-grained supervision and mitigates the credit misassignment issue.

5.5 Training Dynamics of ProcessLight

To investigate how the process design of ProcessLight affects RL training, we analyze the reward dynamics of different training methods. Specifically, we compare ProcessLight with PPO and GRPO on two process-side indicators: average error claims count and average outcome reward. The former indicator measures the quality of intermediate reasoning. Lower values indicate fewer incorrect traffic claims. The latter indicator measures the quality of the final phase decision.

As shown in Figure 4, ProcessLight rapidly reduces the average error-claim count and consistently maintains a lower error level than PPO and GRPO throughout training. This result shows that process-aware reward verification and step-level credit assignment provide more targeted supervision for intermediate traffic reasoning, leading to more stable and reliable reasoning trajectories. ProcessLight also achieves the best average outcome reward among the compared methods. Over the last 10 training steps, ProcessLight obtains an average

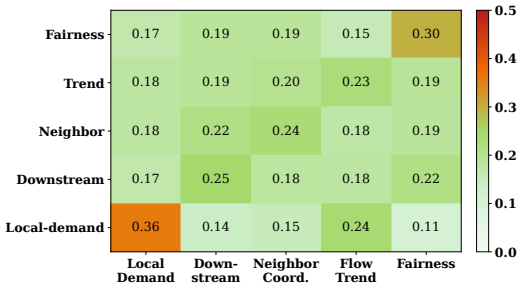


Figure 5: Step importance across traffic regimes. The x-axis denotes reasoning steps, and the y-axis groups traffic regimes by the dominant factor behind the simulator-optimal phase. Each cell shows the normalized importance of a step; higher values indicate a larger change in phase-selection probability after masking that step.

outcome reward of 0.585, compared with 0.574 for GRPO and 0.522 for PPO. These results suggest that improving intermediate reasoning correctness does not compromise final phase selection; instead, more reliable traffic reasoning can further support better control decisions.

5.6 Effectiveness of Step Importance

To verify the effectiveness of our proposed step importance method, we categorize samples into five groups based on the dominant traffic factors that determine the optimal signal phase. We calculate the importance of each reasoning step for all groups and visualize the results via a heatmap. As shown in Figure 5, the cells along the diagonal are significantly brighter, meaning the steps with high importance scores exactly correspond to the core factors affecting traffic efficiency in real scenarios. This result indicates the reliability and effectiveness of our step importance calculation scheme.

6 Conclusion

In this paper, we proposed **ProcessLight**, a process-aware framework for LLM-based traffic signal control. Beyond outcome-only optimization, it decomposes signal decisions into structured traffic reasoning steps and supervises them through step quality reward, step importance, and semantic step tree advantage. Experiments on real-world traffic benchmarks show that ProcessLight achieves the best overall performance and improves most metrics over traditional transportation methods, reinforcement learning controllers, and LLM-based baselines. Ablation and process-supervision analyses further confirm that fine-grained reasoning supervision improves both intermediate reasoning reliability and final phase-decision quality.

Limitations

Firstly, the claim-based verification incurs higher implementation complexity, since it relies on structured traffic observations, predefined reasoning steps, and rule-based matching between generated claims and traffic-state evidence. We will explore learnable verifiers for lower efforts. Secondly, step importance estimation introduces extra training overhead, caused by counterfactual estimation for reasoning steps. More efficient approximation or amortized estimation strategies could improve ProcessLight. Finally, STeP-PO is designed for structured traffic reasoning with predefined semantic steps on standard four-way intersections. The extension of ProcessLight to flexible free-form reasoning formats on more complex three- or five-way intersections (Shen et al., 2024) still remains as important future research direction.

Acknowledgment

This work was supported by the National Key R&D Program of China (Grant No. 2023YFB4301900).

Ethics Statement

We abide by the ACL Code of Ethics. The data resources used in this study are publicly available. During the preparation of this work the authors used ChatGPT in order to polish manuscript language and assist with minor simulation code drafting. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

Chacha Chen, Hua Wei, Nan Xu, Guanjie Zheng, Ming Yang, Yuanhao Xiong, Kai Xu, and Zhenhui Li. 2020. Toward A thousand lights: Decentralized deep reinforcement learning for large-scale traffic signal control. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 3414–3421, New York, NY, USA. AAAI Press.

Luyu Chen, Zeyu Zhang, Haoran Tan, Quanyu Dai, Hao Yang, Zhenhua Dong, and Xu Chen. 2025. Distributional llm-as-a-judge. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems*

2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025.

Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 4299–4307.

DeepSeek-AI. 2026. Deepseek-v4: Towards highly efficient million-token context intelligence. *CoRR*, arXiv:2606.19348.

Xiaolei Di, Yu Xiao, Chao Zhu, Yang Deng, Qinpei Zhao, and Weixiong Rao. 2019. Traffic congestion prediction by spatiotemporal propagation patterns. In *20th IEEE International Conference on Mobile Data Management, MDM 2019, Hong Kong, SAR, China, June 10-13, 2019*, pages 298–303. IEEE.

Jinlan Fu, See-Kiong Ng, Zhengbao Jiang, and Pengfei Liu. 2024. GPTScore: Evaluate as you desire. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6556–6576, Mexico City, Mexico. Association for Computational Linguistics.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, and 175 others. 2025a. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638.

Yiran Guo, Lijie Xu, Ji Liu, Dan Ye, and Shuang Qiu. 2025b. Segment policy optimization: Effective segment-level credit assignment in RL for large language models. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*.

Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron C. Courville, and Nicolas Le Roux. 2025. Vineppo: Refining credit assignment in RL training of llms. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, volume 267 of *Proceedings of Machine Learning Research*. PMLR / OpenReview.net.

P. Koonce, L.A. Rodegerdts, United States. Federal Highway Administration, Turner-Fairbank Highway Research Center, and Kittelson & Associates. 2008. *Traffic Signal Timing Manual*. U.S. Department of Transportation, Federal Highway Administration.

Siqi Lai, Zhao Xu, Weijia Zhang, Hao Liu, and Hui Xiong. 2025. Llmlight: Large language models as traffic signal control agents. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.1, KDD 2025, Toronto, ON, Canada, August 3-7, 2025*, pages 2335–2346, Toronto, ON, Canada. ACM.

Enming Liang, Zicheng Su, Chilin Fang, and Renxin Zhong. 2022. OAM: an option-action reinforcement learning framework for universal multi-intersection control. In *Thirty-Sixth AAAI Conference on Artificial Intelligence, AAAI 2022, Thirty-Fourth Conference on Innovative Applications of Artificial Intelligence, IAAI 2022, The Twelveth Symposium on Educational Advances in Artificial Intelligence, EAAI 2022 Virtual Event, February 22 - March 1, 2022*, pages 4550–4558. AAAI Press.

Afshin Oroojlooy, MohammadReza Nazari, Davood Hajinezhad, and Jorge Silva. 2020. Attendlight: Universal attention-based reinforcement learning model for traffic signal control. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.

Syed Shah Sultan Mohiuddin Qadri, Mahmut Ali Gökçe, and Erdinç Öner. 2020. State-of-art review of traffic signal control methods: challenges and opportunities. *European Transport Research Review*, 12(1):55.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *Preprint*, arXiv:1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *Preprint*, arXiv:2402.03300.

Jiajing Shen, Jiaxing Hu, Qinpei Zhao, and Weixiong Rao. 2024. Heterogeneous traffic intersections control design based on reinforcement learning. *IET Intelligent Transport Systems*, 18(10):1760–1776.

Pravin Varaiya. 2013. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195.

Hua Wei, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash V. Gayah, Kai Xu, and Zhenhui Li. 2019a. Presslight: Learning max pressure control to coordinate traffic signals in arterial network. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2019, Anchorage, AK, USA, August 4-8, 2019*, pages 1290–1298, Anchorage, AK, USA. ACM.

Hua Wei, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu, and Zhenhui Li. 2019b. Colight: Learning network-level cooperation for traffic signal control. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management, CIKM 2019, Beijing, China, November 3-7, 2019*, pages 1913–1922, Beijing, China. ACM.

Hua Wei, Guanjie Zheng, Vikash V. Gayah, and Zhenhui Li. 2019c. A survey on traffic signal control methods. *Preprint*, arXiv:1904.08117.

Hua Wei, Guanjie Zheng, Huaxiu Yao, and Zhenhui Li. 2018. Intellilight: A reinforcement learning approach for intelligent traffic light control. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19-23, 2018*, pages 2496–2505, London, UK. ACM.

Qiang Wu, Mingyuan Li, Jun Shen, Linyuan Lü, Bo Du, and Ke Zhang. 2023. Transformerlight: A novel sequence modeling based traffic signaling mechanism via gated transformer. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD 2023, Long Beach, CA, USA, August 6-10, 2023*, pages 2639–2647. ACM.

Qiang Wu, Liang Zhang, Jun Shen, Linyuan Lü, Bo Du, and Jianqing Wu. 2021. Efficient pressure: Improving efficiency for signalized intersections. *Preprint*, arXiv:2112.02336.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 23 others. 2025. Qwen2.5 technical report. *Preprint*, arXiv:2412.15115.

Bingquan Yu, Jinqiu Guo, Qinpei Zhao, Jiangfeng Li, and Weixiong Rao. 2020. Smarter and safer traffic signal controlling via deep reinforcement learning. In *CIKM '20: The 29th ACM International Conference on Information and Knowledge Management, Virtual Event, Ireland, October 19-23, 2020*, pages 3345–3348. ACM.

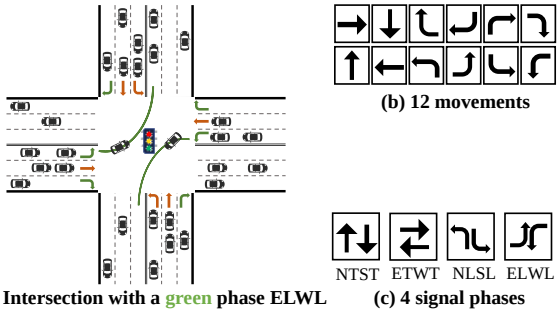
Zirui Yuan, Siqi Lai, and Hao Liu. 2026. CoLLM-Light: Cooperative large language model agents for network-wide traffic signal control. In *The Fourteenth International Conference on Learning Representations*.

Huichu Zhang, Siyuan Feng, Chang Liu, Yaoyao Ding, Yichen Zhu, Zihan Zhou, Weinan Zhang, Yong Yu, Haiming Jin, and Zhenhui Li. 2019. Cityflow: A multi-agent reinforcement learning environment for large scale city traffic scenario. In *The World Wide Web Conference*, WWW '19, page 3620–3624, New York, NY, USA. Association for Computing Machinery.

Jingyi Zhang, Jiaxing Huang, Huanjin Yao, Shunyu Liu, Xikun Zhang, Shijian Lu, and Dacheng Tao. 2025. R1-VL: learning to reason with multimodal large language models via step-wise group relative policy optimization. In *IEEE/CVF International Conference on Computer Vision, ICCV 2025, Honolulu, HI, USA, October 19-25, 2025*, pages 1859–1869, Honolulu, HI, USA. IEEE.

Liang Zhang, Qiang Wu, Jun Shen, Linyuan Lü, Bo Du, and Jianqing Wu. 2022. Expression might be enough: representing pressure and demand for reinforcement learning based traffic signal control. In *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, Proceedings of Machine Learning Research, pages 26645–26654, Baltimore, Maryland, USA. PMLR.

Xingchen Zou, Yuhao Yang, Zheng Chen, Xixuan Hao, Yiqi Chen, Chao Huang, and Yuxuan Liang. 2026. Traffic-r1: Reinforced llms bring human-like reasoning to traffic signal control systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2026, San Diego, California, United States, July 2-7, 2026*, pages 21823–21838. Association for Computational Linguistics.



(a) Intersection with a green phase ELWL (b) 12 movements (c) 4 signal phases

Figure 6: Illustration of (a) intersections, (b) traffic movements, and (c) signal phases.

A Appendix

A.1 Concepts in traffic signal control

We illustrate the commonly used settings in traffic signal control in Figure 6.

A.2 Baseline Methods

We categorize the baselines into traditional methods, reinforcement learning methods, and large language model methods.

A.2.1 Traditional Methods.

- **FixedTime** (Koonce et al., 2008): FixedTime uses predefined cycle lengths, phase orders, and green durations, providing a simple and stable controller that does not adapt to real-time traffic demand.
- **MaxPressure** (Varaiya, 2013): MaxPressure selects the phase with the largest pressure difference between upstream queues and downstream traffic conditions, achieving strong queue dissipation through transportation-theoretic control.

A.2.2 RL-based Methods.

- **PressLight** (Wei et al., 2019a): PressLight applies deep reinforcement learning to optimize intersection pressure, combining adaptive signal control with a congestion-oriented traffic objective.
- **MPLight** (Chen et al., 2020): MPLight builds on FRAP and uses pressure as both observation and reward, enabling pressure-aware phase competition modeling for traffic signal control.
- **AttendLight** (Oroojlooy et al., 2020): AttendLight employs attention mechanisms to construct traffic observations and model phase transition dependencies, allowing the agent to focus on critical movements.
- **CoLight** (Wei et al., 2019b): CoLight uses graph attention networks for communication among neighboring intersections, enabling cooperative multi-agent signal control over road networks.
- **Efficient-CoLight** (Wu et al., 2021): Efficient-CoLight improves CoLight by introducing efficient pressure observations, reducing redundant state information while preserving key coordination signals.
- **Advanced-CoLight** (Zhang et al., 2022): Advanced-CoLight extends CoLight with richer traffic state features such as pressure and effective running vehicles, improving expressive multi-intersection coordination.

A.2.3 LLM-based Methods.

- **LLMLight** (Lai et al., 2025): LLMLight uses an LLM-based traffic signal control agent to interpret traffic states and generate signal actions through natural-language traffic reasoning.
- **Traffic-R1** (Zou et al., 2026): Traffic-R1 develops an LLM-based traffic signal control agent with a two-stage reinforcement learning framework to improve real-world deployment capability.
- **CoLLMLight** (Yuan et al., 2026): CoLLMLight coordinates multiple LLM-based traffic signal control agents with cost-aware reasoning depth, balancing cooperative decision quality and inference overhead.

A.3 Metrics Details

Average Travel Time (ATT). ATT measures the average time required for vehicles to complete their trips within the road network. It directly reflects the overall traffic efficiency under the learned signal control policy. Formally, ATT is defined as

$$\mathrm{ATT} = \frac{1}{N}\sum_{v=1}^{N}\left(t_v^{\mathrm{arrive}} - t_v^{\mathrm{depart}}\right), \qquad (14)$$

where N denotes the total number of vehicles, and t_v^{depart} and t_v^{arrive} denote the departure and arrival times of vehicle v, respectively.

Table 5: Hyperparameter sensitivity of STeP-PO on Jinan1 and Hangzhou1.

#	η	λ	G	Jinan1 ATT ↓	Hangzhou1 ATT ↓
1	0.3	0.2	4	307.87	337.24
2	0.3	0.3	8	291.62	319.33
3	**0.6**	**0.3**	**8**	**289.34**	**314.99**
4	0.6	0.4	8	296.76	320.52
5	0.9	0.3	12	294.48	321.05
6	0.9	0.5	12	302.15	329.41

Average Waiting Time (AWT). AWT measures the average accumulated waiting time of vehicles during the trip. A vehicle is considered waiting when its speed falls below a predefined threshold. Lower AWT indicates reduced intersection delay and smoother traffic flow. It is computed as

$$\mathrm{AWT} = \frac{1}{N}\sum_{v=1}^{N} w_v, \tag{15}$$

where w_v is the total waiting time of vehicle v.

Average Queue Length (AQL). AQL measures the average number of queuing vehicles on all lanes and decision steps, indicating the congestion level of the traffic network. It is defined as

$$\mathrm{AQL} = \frac{1}{T|\mathcal{L}|}\sum_{t=1}^{T}\sum_{l\in\mathcal{L}} q_l^t, \tag{16}$$

where T denotes the total number of simulation steps, $\mathcal{L}$ denotes the set of lanes, and q_l^t represents the queue length of lane l at simulation step t.

A.4 STeP-PO Algorithm

As shown in Algorithm 1, STeP-PO consists of three sequential stages: training data collection, policy warm-up, and step-wise policy optimization.

A.5 Semantic Step Tree Construction

We present the semantic step tree construction algorithm in Algorithm 2.

A.6 Hyperparameters Analysis

We analyze the sensitivity of STeP-PO to three key hyperparameters on Jinan1 and Hangzhou1 using ATT as the evaluation metric. Specifically, η controls the strength of phase-sensitive step importance weighting, λ balances the step-level advantage and outcome-level feedback, and G denotes the number of sampled reasoning trajectories used to construct the semantic step tree. As shown in Table 5, STeP-PO achieves the best performance with $\eta = 0.6$, $\lambda = 0.3$, and $G = 8$, which is used as the default setting in our main experiments. Increasing G from 4 to 8 consistently reduces ATT on both datasets, indicating that more rollouts improve reasoning diversity and provide richer sibling branches for more stable advantage estimation.

The results also show that both λ and η should be set to moderate values. A proper λ improves performance by incorporating step-quality supervision into policy optimization, while a larger value degrades ATT because overemphasizing step-level rewards weakens the supervision from the final phase outcome. Similarly, increasing η from 0.3 to 0.6 improves ATT, demonstrating that reasonable importance weighting helps optimize decision-critical reasoning steps. Nevertheless, when η is further increased to 0.9, performance declines, implying

Algorithm 1 STeP-PO

Require: Policy model π_θ initialized by a pre-trained LLM; teacher model π_{tea}; traffic simulator $\mathcal{S}$; traffic scenarios $\mathcal{C}$; group size G

Ensure: Trained traffic signal control policy π_θ

Training data collection:

Collect traffic scenarios from $\mathcal{S}$ and construct input prompts following Section 4.1

Select oracle phase actions through network-level simulation and collect teacher-generated step-wise reasoning trajectories

Build dataset $\mathcal{D} = \{(x, y)\}$ and split it into $\mathcal{D}_{\mathrm{SFT}}$ and $\mathcal{D}_{\mathrm{RL}}$

Policy warm-up:

for each sample $(x, y) \in \mathcal{D}_{\mathrm{SFT}}$ **do**

    Optimize π_θ with the supervised warm-up objective in Eq. 2

end for

Set reference policy $\pi_{\mathrm{ref}} \leftarrow \pi_\theta$

Step-wise policy optimization:

for each sample $(x, y^\star) \in \mathcal{D}_{\mathrm{RL}}$ **do**

    Generate a group of reasoning trajectories $\{y_i\}_{i=1}^{G} \sim \pi_\theta(\cdot|x)$

    Compute step quality rewards using the reference trajectory $y^\star$ by Eq. 3

    Estimate phase-sensitive step importance by Eq. 5

    Construct the semantic step tree and compute step advantages by Eq. 9

    Optimize π_θ with the STeP-PO objective in Eq. 12

end for

return π_θ

Algorithm 2 Semantic Step Tree Construction

Require: Input prompt x; sampled trajectories $\{y_i\}_{i=1}^{G}$, where $y_i = (s_i^1, \ldots, s_i^K)$; final action of each trajectory a_i.

Ensure: Semantic step tree $\mathcal{T}$.

Initialize $\mathcal{T}$ with a root node n_0 representing the input prompt x.
for $i = 1$ to G **do**
  Set the current node $n \leftarrow n_0$.
  for $k = 1$ to K **do**
    Extract the claim set C_i^k from reasoning step s_i^k.
    Search among the children of n for a node n' at step k whose claim set is semantically matched with C_i^k.
    if such a node n' exists **then**
      Merge s_i^k into n' and add it to the step set $\mathcal{S}(n')$.
      Set $n \leftarrow n'$.
    else
      Create a new child node n' under n.
      Store C_i^k, s_i^k, and its step index k in n'.
      Set $\mathcal{S}(n') = \{s_i^k\}$ and $n \leftarrow n'$.
    end if
  end for
  Attach the final action a_i as a leaf node under n.
end for
return $\mathcal{T}$.

that excessive reliance on a few highly sensitive steps may ignore the contribution of the complete reasoning trajectory. These results confirm that STeP-PO benefits from a balanced combination of rollout diversity, step-quality supervision, and phase-sensitive credit allocation.

A.7 Statistical Significance Analysis

Each model is evaluated using five random seeds (11, 42, 1024, 2026, 10001) on Jinan1 and Hangzhou1. Table 6 reports the mean and standard deviation across seeds. ProcessLight achieves the best mean performance on most metrics, and its coefficient of variation ranges from 0.44% to 9.76%, indicating limited sensitivity to random-seed variation.

To further assess robustness, we conduct two-sided Welch's t-tests comparing each baseline against ProcessLight and apply one global Holm correction across all 90 comparisons. ProcessLight obtains a lower mean in 88 of the 90 comparisons; 81 comparisons significantly favor ProcessLight after correction, 9 show no detectable difference, and none significantly favors a baseline. The nine non-significant comparisons involve AdvancedCoLight on Jinan1 (ATT) and Hangzhou1 (AWT/AQL), LLMLight on Jinan1 (ATT/AWT) and Hangzhou1 (AWT), and Traffic-R1 on Jinan1 (ATT/AWT) and Hangzhou1 (AWT). Although AdvancedCoLight achieves lower mean AWT and AQL on Hangzhou1, neither difference remains statistically significant after Holm correction.

A.8 Effectiveness of Step Quality

To directly compare step-quality supervision with outcome-only supervision, we sample 100 traffic states and generate reasoning trajectories using Qwen2.5-3B. We then inject two types of controlled noise into each trajectory: logically reversing observation-consistent claims, and replacing the final action with a valid but verified suboptimal phase. DeepSeek-V4-Pro is used as the judge model (Fu et al., 2024) to label the correctness of each reasoning step, which determines the ground-truth sign of its advantage. Outcome-only supervision assigns the trajectory-level advantage sign to all reasoning steps, whereas Step Quality augments the trajectory-level signal with step-specific validity rewards.

Table 7 reports the accuracy of the resulting advantage signs under both methods. Outcome-only supervision assigns an incorrect optimization direction to 22% of the reasoning steps, whereas Step Quality correctly identifies all injected intermediate errors. This indicates that Step Quality enables STeP-PO to distinguish reliable reasoning steps from hallucinated ones and assign them correct optimization directions.

A.9 Reliability of Claim Verification

The step-quality evaluation pipeline consists of three stages: regular-expression parsing of the generated claims, soft semantic matching against the reference key claims (Zhang et al., 2025), and observation-based validity checking. To quantify the reliability of each stage, we randomly sample 100 instances, roll out 8 responses for each prompt, and conduct three independent evaluations using DeepSeek-V4-Pro as the judge model (Chen et al., 2025). As shown in Table 8, all three stages achieve high execution accuracy, supporting the reliability

Table 6: Mean and standard deviation over five random seeds on Jinan1 and Hangzhou1. Table 2 reports results under the default seed. Best results are in bold.

Method	Jinan 1			Hangzhou 1		
	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓
Traditional Methods						
FixedTime	409.73 ± 0.00	156.76 ± 0.00	190.07 ± 0.00	564.86 ± 0.00	204.63 ± 0.00	169.56 ± 0.00
MaxPressure	305.26 ± 0.00	60.19 ± 0.00	72.97 ± 0.00	343.94 ± 0.00	47.09 ± 0.00	39.02 ± 0.00
RL Methods						
EfficientPressLight	316.85 ± 0.00	77.93 ± 0.00	136.27 ± 0.00	472.22 ± 0.01	178.99 ± 0.00	148.31 ± 0.00
EfficientCoLight	310.22 ± 0.01	70.10 ± 0.00	122.57 ± 0.00	340.52 ± 0.00	43.85 ± 0.00	36.34 ± 0.00
AttendLight	514.56 ± 21.96	251.71 ± 16.56	440.15 ± 28.95	481.44 ± 6.99	178.86 ± 7.56	148.21 ± 6.26
EfficientMPLight	311.36 ± 0.07	71.33 ± 0.14	124.72 ± 0.26	384.46 ± 0.01	93.46 ± 0.00	77.44 ± 0.00
AdvancedCoLight	298.81 ± 0.00	60.40 ± 0.00	105.61 ± 0.00	328.58 ± 0.00	**32.95 ± 0.00**	**27.30 ± 0.00**
MPLight	489.20 ± 15.31	233.69 ± 13.35	408.62 ± 23.34	517.53 ± 15.94	208.65 ± 13.73	172.89 ± 11.38
AdvancedMPLight	313.15 ± 0.00	75.18 ± 0.00	131.47 ± 0.00	372.21 ± 0.01	80.23 ± 0.00	66.48 ± 0.00
CoLight	474.29 ± 21.28	215.33 ± 16.43	376.53 ± 28.73	485.46 ± 9.60	175.46 ± 6.92	145.38 ± 5.73
LLM-based Methods						
LightGPT-3B	356.32 ± 4.60	110.68 ± 4.10	134.21 ± 4.97	378.87 ± 3.23	82.81 ± 2.49	68.62 ± 2.07
LightGPT-8B	337.62 ± 1.15	94.79 ± 1.16	114.93 ± 1.41	395.63 ± 2.85	105.55 ± 2.84	87.45 ± 2.35
LLMLight	297.58 ± 0.24	52.69 ± 0.29	63.89 ± 0.35	337.04 ± 0.53	41.38 ± 0.37	34.29 ± 0.31
CoLLMLight	303.56 ± 1.14	59.05 ± 1.11	71.61 ± 1.35	342.71 ± 1.09	47.86 ± 0.90	39.66 ± 0.75
Traffic-R1	299.13 ± 0.45	54.01 ± 0.40	65.49 ± 0.49	336.56 ± 0.82	40.04 ± 0.73	33.17 ± 0.61
ProcessLight (ours)						
Qwen2.5-3B	335.26 ± 3.74	90.49 ± 3.69	109.72 ± 4.48	359.00 ± 1.20	64.67 ± 1.60	53.59 ± 1.32
Qwen3-8B	308.79 ± 5.49	74.75 ± 2.74	83.14 ± 2.04	342.45 ± 7.88	45.53 ± 3.45	37.16 ± 2.53
Qwen3-32B	310.31 ± 1.28	64.20 ± 1.17	77.84 ± 1.42	339.53 ± 0.95	42.52 ± 0.80	35.23 ± 0.67
DeepSeek-V4-Flash	297.53 ± 5.09	53.92 ± 4.35	63.61 ± 3.79	323.84 ± 3.43	45.77 ± 4.49	35.43 ± 3.52
DeepSeek-V4-Pro	291.31 ± 2.74	54.65 ± 2.84	63.62 ± 2.87	325.56 ± 3.50	43.21 ± 2.75	35.30 ± 3.38
ProcessLight	**290.55 ± 4.50**	**46.31 ± 4.52**	**53.13 ± 1.17**	**312.61 ± 1.38**	35.66 ± 3.27	29.27 ± 1.14

Table 7: Accuracy of step-level advantage signs under controlled noise injection.

Supervision	Advantage-Sign Accuracy
Outcome-only	78%
With Step Quality	**100%**

Table 8: Execution accuracy of the three verification stages.

Component	Execution Accuracy
Claim extraction	99.81 ± 0.02%
Semantic matching	96.87 ± 0.04%
Observation validity	97.25 ± 0.46%

of the step-quality signal for policy optimization.

A.10 Generalization across Backbones and Teachers

We further evaluate ProcessLight with Qwen3-8B and Llama3-8B as backbone models, and with Qwen3-Plus as the teacher model for training-data collection. Table 11 reports the zero-shot performance across the five real-world datasets. ProcessLight remains effective across the tested settings: all three backbone variants outperform LLMLight, CoLLMLight, and Traffic-R1 on most metrics, and the Qwen3-8B variant improves over the Qwen2.5-3B variant on 12 of the 15 metrics. These results provide initial evidence that the method is not tied to a single backbone family or to the original teacher model; our conclusions are delimited to the models evaluated here.

A.11 Restricted-Observation Comparison

To isolate the contribution of the training strategy from the richer observation, we remove neighboring-intersection information and the corresponding reasoning guidance from the ProcessLight input, and evaluate the modified model on all five datasets. As shown in Table 10, the restricted variant still outperforms LLMLight on most metrics. We note that the restricted-observation experiment serves as a fairness control that isolates the contribution of the training strategy, whereas the full model reflects the intended

Table 9: Cost of the two data-construction stages.

Stage	Model / Environment	Calls	Token Usage	Runtime	Cost
Reference claims	DeepSeek-V4-Pro, Cloud API	3k	7.07M in + 1.70M out	–	$9.1
Step importance	Qwen2.5-3B, 1× RTX PRO 6000	21k	57.81M in; no output	1.5 GPU-h	$1.2

Table 10: Zero-shot performance under restricted observation without neighboring-intersection information. Best results are in bold.

Method	Jinan 1			Jinan 2			Jinan 3			Hangzhou 1			Hangzhou 2		
	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓
LLMLight	**297.28**	52.28	**63.39**	310.48	70.66	123.56	**291.86**	**55.96**	85.40	337.21	41.60	34.47	419.60	84.91	**164.73**
ProcessLight w/o Neighbor Info.	300.91	**48.87**	65.63	**304.32**	**68.35**	**119.47**	299.48	56.23	**81.59**	**332.06**	**39.14**	**34.38**	**414.50**	**83.65**	168.67

Table 11: Zero-shot performance with different backbones and teacher models. Best results are in bold.

Method	Teacher	Jinan 1			Jinan 2			Jinan 3			Hangzhou 1			Hangzhou 2		
		ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓	ATT↓	AWT↓	AQL↓
LLMLight	–	297.28	52.28	63.39	310.48	70.66	123.56	291.86	55.96	85.40	337.21	41.60	34.47	419.60	84.91	164.73
CoLLMLight	–	298.99	54.35	65.90	311.23	71.06	124.25	292.25	56.58	86.35	336.25	41.30	34.22	419.96	74.02	**143.59**
Traffic-R1	–	299.14	54.15	65.65	310.79	70.54	123.34	291.96	56.06	85.56	337.46	40.94	33.92	421.28	81.03	157.20
ProcessLight (Qwen2.5-3B)	DeepSeek-V4-Pro	289.34	45.03	55.94	**293.18**	**54.16**	102.67	**284.33**	50.97	78.20	314.99	32.82	30.48	407.24	73.13	154.21
ProcessLight (Qwen3-8B)	DeepSeek-V4-Pro	**288.81**	**43.18**	54.49	295.60	54.57	100.39	285.15	**48.43**	**77.18**	306.95	**30.98**	**30.13**	**401.47**	**71.06**	**152.25**
ProcessLight (Llama3-8B)	DeepSeek-V4-Pro	289.98	46.35	**52.57**	299.28	63.17	**96.90**	294.29	55.03	82.13	**306.15**	35.29	38.96	409.45	72.54	156.53
ProcessLight (Qwen2.5-3B)	Qwen3-Plus	290.73	47.66	56.29	296.88	67.42	105.32	301.74	56.50	87.26	309.72	37.05	45.56	410.15	79.60	152.62

network-coordination problem setting.

A.12 Computational Cost Analysis

Prior to RL optimization, we precompute the step-importance scores for each prompt and reuse them across all rollouts associated with that prompt; compared with GRPO, step-importance estimation therefore introduces no additional model forward or backward passes during training. For N prompts with K reasoning steps, generating the reference claims requires N calls to the teacher model, while step-importance estimation requires $N(K+1)$ calls to the SFT-initialized policy model—one for the complete response and K for the step-masked variants. Step-importance estimation thus accounts for $\frac{K+1}{K+2}$ of all model calls, i.e., 87.5% with $K = 6$. This call-count proportion overestimates the practical overhead, because these calls use a locally deployed 3B model for forward-only scoring, without autoregressive decoding or additional traffic simulation. As shown in Table 9, step-importance estimation accounts for only 11.6% of the combined monetary cost of the data-construction pipeline.

A.13 Prompt Template

```
You are an expert traffic signal control agent operating under a strict reasoning framework.
Your task is to select exactly one legal phase for the current intersection based on the provided traffic facts.

## Background Context
You are a traffic signal controller at a four-way intersection with 12 movements:
[NL, NT, NR, SL, ST, SR, EL, ET, ER, WL, WT, WR] and 4 phases
[ETWT, NTST, ELWL, NLSL]. Each phase releases two compatible movements:
ETWT releases ET and WT, NTST releases NT and ST, ELWL releases EL and WL,
and NLSL releases NL and SL.

Legal phases for this decision: [ETWT, NTST, ELWL, NLSL].

## Decision Rules
1. Prioritize local intersection pressure relief; grant right-of-way first to long queues and stop-line nearby vehicles.
2. Assess downstream congestion before releasing any movement. Avoid discharging vehicles into congested downstream sections.
3. Use adjacent intersections only as local risk reference in this decision step.
4. Avoid prolonged single-lane waiting when downstream conditions permit.
5. Occupancy 0-0.2 indicates free flow, 0.2-0.4 basically smooth, above 0.4 mild congestion, and above 0.6 severe congestion.

## Current Intersection State

### Movement State

| Movement | Phase | Early Queue Length | Downstream Queue Length | Pressure | Avg Waiting Time(minutes) | Segment Veh Occupancy |
      Vehicle Count |
| --- | --- | --- | --- | --- | --- | --- | --- |
| ET | ETWT | 0 | 10 | -3.3 | 0.1 | [0, 0.2, 0] | 3 |
| WT | ETWT | 0 | 2 | -0.7 | 0 | [0, 0.1, 0] | 2 |
| NT | NTST | 1 | 0 | 1 | 0.1 | [0, 0.1, 0.1] | 9 |
| ST | NTST | 1 | 5 | -0.7 | 0 | [0.1, 0, 0.1] | 10 |
| EL | ELWL | 4 | 0 | 4 | 0.3 | [0, 0, 0.1] | 6 |
| WL | ELWL | 2 | 5 | 0.3 | 0.8 | [0, 0, 0] | 2 |
| NL | NLSL | 2 | 2 | 1.3 | 0.6 | [0, 0, 0] | 3 |
| SL | NLSL | 2 | 10 | -1.3 | 0.2 | [0, 0, 0] | 4 |

### Phase Aggregation

Phase ranking by aggregated signal pressure: ELWL > NTST > NLSL > ETWT
Highest pressure phase from computed indicators: ELWL

| Phase | Movement | Early Queue Length | Downstream Queue Length | Pressure | Avg Waiting Time(minutes) | Segment Veh Count |
     Vehicle Count |
| --- | --- | --- | --- | --- | --- | --- | --- |
| ETWT | ET,WT | 0 | 12 | -4 | 0 | [0, 3, 2] | 5 |
| NTST | NT,ST | 2 | 5 | 0.3 | 0 | [1, 4, 12] | 19 |
| ELWL | EL,WL | 6 | 5 | 4.3 | 0.4 | [0, 0, 2] | 8 |
| NLSL | NL,SL | 4 | 12 | 0 | 0.4 | [0, 0, 3] | 7 |

## Adjacent Intersection Context

| Direction | Intersection ID | ET | WT | NT | ST | EL | WL | NL | SL | Related Movements |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| East | intersection_1_3 | 11, 0.5 | 0, 0.1 | 4, 0.1 | 14, 0.2 | 10, 0.2 | 2, 0 | 4, 0.1 | 3, 0.1 | ET,SL -> ET,EL |
| West | intersection_3_3 | 6, 0.2 | 1, 0 | 0, 0.1 | 0, 0.1 | 4, 0.1 | 2, 0 | 0, 0 | 3, 0.1 | WT,NL -> WT,WL |
| North | intersection_2_2 | 5, 0.2 | 5, 0.2 | 0, 0.1 | 0, 0.1 | 1, 0 | 2, 0.1 | 2, 0 | 5, 0.1 | NT,EL -> NT,NL |

## Recent History

Recent phases: NLSL -> ETWT -> NTST -> ETWT
Last phase: ETWT

| Phase | Queue Change | Pressure Change | Waiting Change | Service Gap | Trend Summary |
| --- | --- | --- | --- | --- | --- |
| ETWT | 6 -> 0 -> 7 -> 0 | 5.7 -> -2.3 -> 4.3 -> -4 | 0.2 -> 0 -> 0.1 -> 0 min | 0 steps | recently served and clearly relieved |
| NTST | 7 -> 11 -> 0 -> 2 | 5.3 -> 9.3 -> -2.7 -> 0.3 | 0.3 -> 0.4 -> 0 -> 0 min | 1 steps | recently served and clearly relieved
      |
| ELWL | 1 -> 2 -> 3 -> 6 | -0.7 -> 0.3 -> 0.3 -> 4.3 | 0.1 -> 0.2 -> 0.3 -> 0.4 min | 5 steps | unserved for long and demand is
     worsening |
| NLSL | 0 -> 2 -> 4 -> 4 | -0.3 -> -0.3 -> 1.3 -> 0 | 0 -> 0.1 -> 0.2 -> 0.4 min | 3 steps | demand is increasing |

## Output Prompt
Reason step-by-step via seven fixed Level-2 heading sections:
Phase Mapping, Local Demand, Downstream, Neighbor Coordination, Flow Trend, Fairness, and Final Answer.

Each reasoning section should contain no more than three atomic <claim> tags.
Only mathematically derivable conclusions following the unified syntax may be placed within <claim>.
The final selected phase must be enclosed by <phase></phase>.
```

Table 12: The prompt template

```
## Phase Mapping
Phase mapping links each valid signal phase to its corresponding traffic movements. The phases are ranked by aggregated pressure
      to identify priority.
<claim>RANK[Pressure][Phase]=ELWL > NTST > NLSL > ETWT</claim>
<claim>MAX[Early Queue Length][Phase]=ELWL</claim>
<claim>MAX[Avg Waiting Time][Phase]=ELWL</claim>

## Local Demand
ELWL exhibits the highest local demand with an aggregated pressure of 4.3 and an early queue length of 6, both exceeding all other
       phases. WL's average waiting time of 0.8 minutes is the highest among movements.
<claim>[ELWL][Pressure]=4.3</claim>
<claim>[ELWL][Early Queue Length]=6</claim>
<claim>[WL][Avg Waiting Time]=0.8</claim>

## Downstream
Several downstream segments show high queue lengths that could limit discharge. SL and ET each have a downstream queue of 10,
      while WL has a downstream queue of 5, indicating potential congestion but still within manageable limits for ELWL's left-
      turn movement.
<claim>[SL][Downstream Queue Length]=10</claim>
<claim>[ET][Downstream Queue Length]=10</claim>
<claim>[WL][Downstream Queue Length]=5</claim>

## Neighbor Coordination
Adjacent intersections provide context for inflow and outflow. West-bound WL has a low queue of 2, allowing discharge. East-bound
      ET shows a high occupancy of 0.5, suggesting some downstream constraint, and North-bound NT has low occupancy of 0.1.
<claim>ADJ[West][WL][Queue]=2</claim>
<claim>ADJ[East][ET][Occupancy]=0.5</claim>
<claim>ADJ[North][NT][Occupancy]=0.1</claim>

## Flow Trend
ELWL has been unserved for 5 steps with worsening demand and increasing pressure and queue, indicating urgent need for service.
<claim>TREND[ELWL][Service Gap]=5 steps (unserved)</claim>
<claim>TREND[ELWL][Pressure Change]=increasing</claim>
<claim>TREND[ELWL][Queue Change]=increasing</claim>

## Fairness
WL holds the maximum average waiting time among all movements, but no movement exceeds the 10-minute threshold. The service gap
      ranking confirms ELWL has been waiting longest.
<claim>MAX[Avg Waiting Time][Movement]=WL</claim>
<claim>No [Movement] has [Avg Waiting Time]>=10</claim>
<claim>RANK[Service Gap][Phase]=ELWL > NLSL > NTST > ETWT</claim>

## Final Answer
<phase>ELWL</phase>
```

Table 13: Example of ProcessLight response.